

Real-Time Adaptive Music Recommendation System

ECE-GY 9183 Machine Learning Systems Engineering and Operations Project Proposal

March 2, 2026

Team Members and Roles

- Serving: **Vishal Jha**
- Data: **Yonghyeon Shin**
- Training: **Dawei Sun**

3.1 Public-Facing Service (1/2)

What Service Are We Building?

- A **personal music server** where users build and manage their own music library.
- Stream tracks to a web/mobile player.
- Realized using **Navidrome**.
- The ML feature is complementary: it recommends what to play next.

Why a Personal Music Server?

- Designed for users who **value privacy** and avoid mass-market tracking.
- Supports users who already curate their own music collections.
- No dependence on large centralized recommendation ecosystems.
- Provides a realistic but controlled setting for personalized ML.

3.1 Public-Facing Service (2/2)

Audience, Context, and Scale Assumptions

- **Primary audience:** students and hobbyists self-hosting personal libraries.
- Small groups sharing a server.
- Recommendations triggered mainly at **song end** or **skip**.
- **Design scale:** 100 concurrent listeners for load tests (demo < 20).

Open-Source Base and Suitability

- Mature OSS with a realistic playback loop.
- Clear entity IDs + metadata pipeline for event impression logging.
- Fully hostable on Chameleon; recommender integrates via API/plugin hooks.

3.2 Complementary ML Feature Design(1/2)

Problem Statement and Intended Outcome

- **Problem:** Self-hosted music libraries rely on manual search or static playlists; adaptive personalization is limited.
- **Outcome:** After each end/skip event, the system returns **Top- K next tracks** and updates the user's queue, adapting to:
 - short-term session mood
 - long-term user taste

3.2 Complementary ML Feature Design (2/2)

Inputs → Model → Outputs

- **Inputs:** recent session songs (last 1–5), user history aggregates, context (time-of-day), candidate song metadata/embeddings.
- **Model outputs:** Top- K recommended **song IDs** (optionally with scores + model version).
- **System action:** write IDs into the **Up Next queue** (target length 5) with constraints:
 - no repeats
 - filter dislikes
 - basic diversity enforcement

3.3 External Dataset and Data Lineage

Primary Dataset: MTG-Jamendo

- Public research dataset released by the **Music Technology Group (MTG), Universitat Pompeu Fabra**.
- Collected from the Jamendo music platform under Creative Commons licenses.
- Designed for automatic music tagging and recommendation research.
- Contains **55,000+ full-length audio tracks** with **195 curated tags**.

Alignment with Proposed Service

- Audio assets are directly playable in our self-hosted Navidrome service.
- Tag annotations enable pseudo-labeling for cold-start ranking.
- Embeddings extracted from audio support real-time retrieval.

Bogdanov et al., MTG-Jamendo Dataset, Music Technology Group, Universitat Pompeu Fabra.

3.4 Feedback Loop and Retraining

Feedback Captured in Production

- **Implicit signals:** play/start, listen duration (% complete), skip (early vs late), repeat, manual search+play, queue remove/add.
- **Explicit signals:** like/dislike (optional: “more like this” / “not in mood”).

Label Construction

- Positive = like OR repeat OR completion $> 80\%$.
- Negative = dislike OR early skip $< 20\text{--}30\text{s}$.
- Neutral = partial listen.

Retraining Strategy

- Training examples: (session/user features, candidate features, impression list, outcome) → ranking dataset.
- Periodic retraining (nightly or weekly at project scale).
- Triggered retraining if KPI drift (e.g., skip-rate increase) or fallback spikes.

4.1 Two-Stage Recommender Architecture

Stage 1: Embedding-Based Recall

- Pretrained PANNs encoder extracts audio embeddings.
- Retrieve Top-100 nearest neighbors using vector similarity.
- Reduces search space and ensures semantic consistency.

Stage 2: Trainable Ranking Model

- MLP ranker scores candidate tracks.
- Outputs engagement probability.
- Top- K selected with diversity policy ($4+1$).

Loss: Binary Cross Entropy (BCE)

4.2 Cold-Start Data Synthesis Strategy

Challenge

- No real user interaction logs at deployment start.
- Supervised learning not directly feasible.

Virtual Session Generation

- Randomly sample a track a as session context.
- Run Stage 1 retrieval to obtain Top-100 candidates.
- Compute synthetic supervision signal for each candidate.
- Repeat across many seeds (up to enough samples).

Goal

- Approximate online candidate distribution.
- Pre-warm ranker before real feedback is available.

4.3 Pseudo-Label Scoring and Evolution

Semantic Consistency Assumption

Users are more likely to continue listening when stylistic similarity remains high.

Tag Similarity (Jaccard)

$$J_G(a, i) = \frac{|G(a) \cap G(i)|}{|G(a) \cup G(i)|} \quad J_M(a, i) = \frac{|M(a) \cap M(i)|}{|M(a) \cup M(i)|}$$

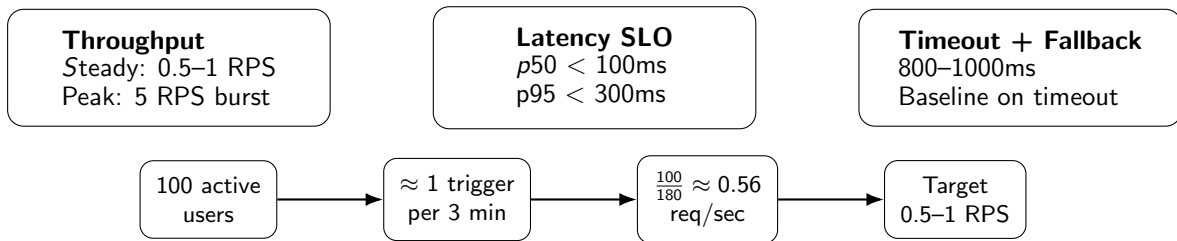
$$y(a, i) = \text{clip}(\alpha J_G + (1 - \alpha) J_M, 0, 1)$$

$$y'(a, i) = \text{clip}(y(a, i) - \beta R(i) - \gamma A(i), 0, 1)$$

Production Transition

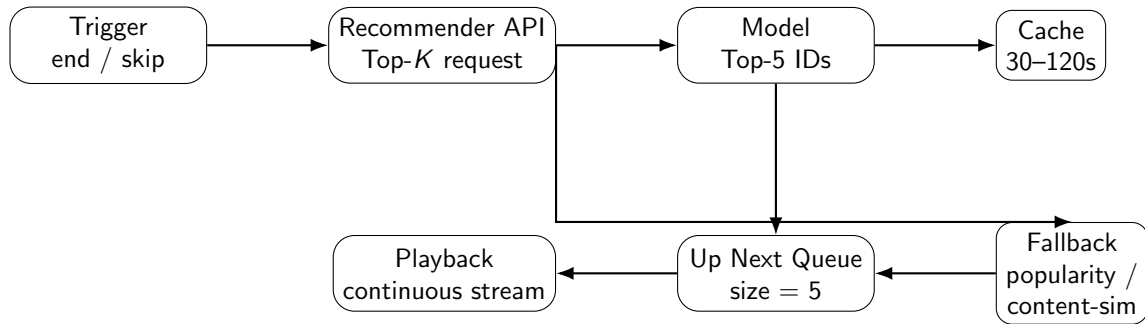
$$y_{real} = \begin{cases} 1 & \text{if END} \\ 0 & \text{if SKIP} \end{cases}$$

5.1 Operational Requirements (Visual Summary)



- Triggering only on **end/skip** keeps load low and predictable.
- Queue buffer ensures **no playback gaps** even if the model is slow.

5.2 Model Output → System Outcome (Visual Pipeline)



Measured outcomes:

skip-rate completion replay/like fallback-rate latency/RPS

- Output contract: **Top-5 track IDs** (+ scores/model version optional).
- System action: write to queue; **never block playback**.

6.1 Data Flow (1/2)

Real-Time Inference Path

- **Playback trigger:** END / SKIP event sent with `user_id`, `session_id`, recent tracks.
- **Context construction:** session window (last 1–5 songs), user aggregates (skip-rate, completion-rate), time-of-day.
- **Two-stage inference:** retrieval (embedding similarity) → ranking model.
- **Top-5 IDs returned** to “Up Next” queue (size = 5).
- **Structured logging:** impression list, chosen track, listen duration, source (RECOMMENDED vs MANUAL), `model_version`.

6.1 Data Flow (2/2)

Training Data Flow

- First-party logs stored in Postgres (events + impressions).
- Raw events converted into labels (positive / negative / neutral).
- Time-based dataset generation (no future leakage).
- Offline retraining + updated model artifacts deployed to serving.

6.2 Candidate Selection and Leakage Avoidance (1/2)

Candidate Selection

- **Stage 1: Retrieval** via content embeddings (MTG-Jamendo pretraining + learned user/session embeddings).
- Top ~ 50 –200 nearest neighbors form candidate pool.
- Ranker scores candidates $\rightarrow$ Top-5 selected.
- Training negatives sampled from shown-but-not-chosen impressions.

Leakage Prevention

- **Time-based split:** train on past, validate/test on future.
- Ensure labels occur **after** recommendation exposure.
- Log `model_version` to isolate evaluation from retraining.

6.2 Candidate Selection and Leakage Avoidance (2/2)

Cold Start Strategy

- Offline content embeddings from **MTG-Jamendo**.
- Onboarding (optional): like/dislike ~ 20 songs.
- Gradual shift from content-similarity $\rightarrow$ personalized embeddings.

7.1 Model Freshness and Automation

Retraining Frequency

- **Scheduled retraining:** Cannot decide yet. It totally depends on the user interactions. While we are not certain that how many user interactions we gonna have .
- **Trigger-based retraining:**
 - skip-rate increase beyond threshold,
 - abnormal fallback/timeout spikes,
 - significant increase in interaction volume.
- Music embeddings are updated only when new music is added in bulk.

7.2 Scaling Requirements

Typical Usage Assumptions

- Designed for **100 concurrent listeners**. A good assumption for a course project.
- Event-triggered inference (mainly end/skip).
- Estimated steady load: **0.5–1 RPS** (estimated as $100 \times \frac{1}{180} \approx 0.56$ RPS, assuming one trigger per user every 3 minutes).

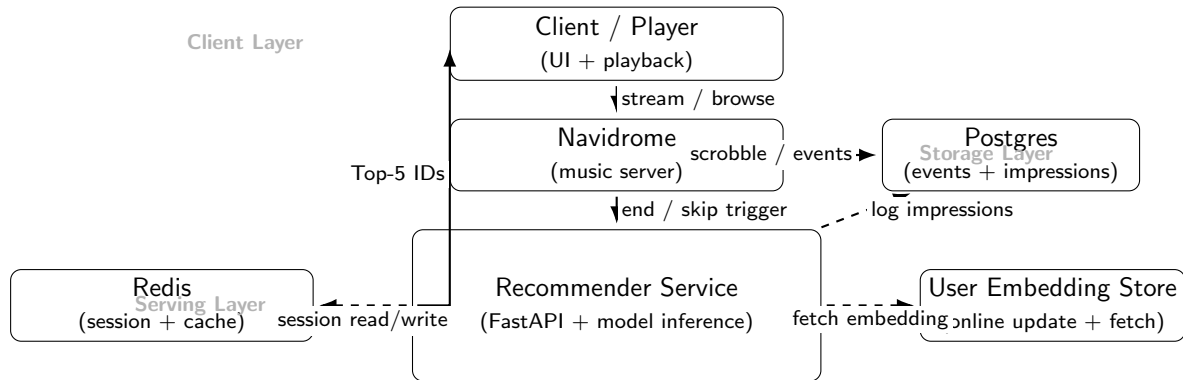
Peak Usage Handling

- Burst assumption: up to **5 RPS** (skip storms + synchronized endings).
- Short-TTL session caching (30–120s) to reduce repeated scoring.
- Timeout (800–1000ms) with fallback to ensure uninterrupted playback.

Right-Sizing Strategy

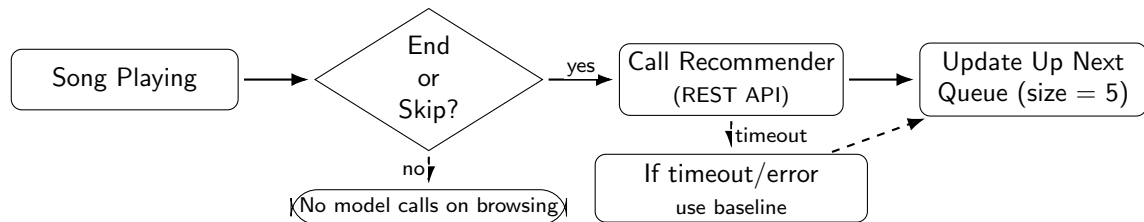
- Start with single CPU-based serving instance.
- Monitor p95 latency, CPU utilization, fallback rate.
- Scale out serving layer if $p95 > 300\text{ms}$ or $\text{CPU} > 70\%$ under load.

8.1 System Flow (Bird's Eye View)



User playback triggers inference; events and impressions are durably logged for retraining.

8.2 Inference Trigger Decision Flow



Only **end/skip** events trigger inference. This limits load, keeps behavior predictable, and aligns model calls with the moments when next-track choice matters.

Questions